

1. A* Search. 3p

Para el caso original donde encontramos las piezas [7,0,2] [7,4,6], la lista devuelta es

1. [[[7, 0, 2], [7, 4, 6]],
2. [[0, 0, 2], [7, 4, 6]],
3. [[6, 4, 6], [0, 0, 2]],
4. [[5, 4, 6], [0, 0, 2]],
5. [[4, 4, 6], [0, 0, 2]],
6. [[3, 4, 6], [0, 0, 2]],
7. [[2, 4, 6], [0, 0, 2]]]

La función $h()$ que implemente basado en los movimientos necesarios de torre y rey blanco para llegar al jaque mate de la posición comparado con las posiciones de jaque mate.

Para el rey es directo, porque para saber cuantos movimientos faltan, como el rey avanza de 1 en 1 simplemente restamos los ejes x e y, sumamos y obtenemos casillas, que es igual a movimientos faltantes para el rey.

Para torre siempre será 0, 1 o 2, debido a que la torre puede avanzar la fila o columna entera. Si ya está en posición de jaque, es 0, si comparte fila o columna con la posición de jaque, los movimientos faltantes es 1, si no cumple ninguna es 2.

Como la torre tiene diferentes posiciones para jaque, comparamos la posición actual con todas las posiciones de jaque y los añadimos a una lista para luego devolver en $h()$ el mínimo de movimientos que se debe hacer.

Para la función A*. He creado mi propio diccionario auxiliar para gestionar los costes de cada movimiento de casilla. Primer paso, pongo en el dictPath el enlace del movimiento anterior, que al ser el state inicial no hay, asigno none y -1.

El coste del estado inicial es 0.

Mientras frontier exista, primero a falta de heap como usan otros ejemplos de código A* uso `sort()` ordenado por la devolución de $h()$ para sacar el teórico más cercano al estado deseado.

Si el estado ya ha sido visitado anteriormente saltamos el proceso, sacamos al "padre" del estado actual y si no es None, es decir, el padre existe, hacemos el movimiento y enlazamos mediante el propio método `movePieces` el estado actual para reflejarlo en el chess.

Compruebo si el estado actual es jaque mate, si es, reconstruimos el camino hecho y salimos de la iteración.

Si no, guardamos en `listVisitedStates` el estado actual, para que el código sepa que ya está visto este estado, como no es el final, sacamos los posibles movimientos a partir del estado actual.

Iteramos sobre esos movimientos posibles, como son futuros movimientos, al recorrido actual le sumamos 1 y lo guardamos en el nuevo coste de este posible movimiento, comparamos para llegar a ese movimiento (si existe, sino será comparado con infinito) si el que estamos analizando ahora es menor a un camino ya hecho, actualizamos valores. sacamos el coste + h y se guarda en el frontier.

Y se itera de nuevo hasta completar el camino, y sigue la regla de A* porq para decidir los pasos siguientes es mediante el frontier el valor primero que es el coste + h

Now, initialize the board with the white king on position [7,7] and rerun your code.

2. Does your algorithm work in precisely the same way? Did you have to change anything? 3p

Al volver a ejecutar el código con la modificación de la posición, el código fallo, se hicieron demasiadas llamadas y nunca encontraba la solución.

Una curiosidad observada fue que implemente para poder ver el tablero en cada iteracion para ver que pasaba y viendo cuando el codigo fallaba por demasiadas llamadas, el tablero el rey negro desaparecia porque era comido por el rey blanco y la torre blanca se movia continuamente emientras el rey seguia en esa posicion.

Otra comprobación que hice fue imprimir los estados en cada iteración y descubrí que por lo que observe y creo comprender.

De primeras observó que el código analiza dos rutas de exploración donde el que se mueve es el rey y otra en la que se mueve es la torre como principal. Tiene sentido porque analiza los posibles movimientos

salida de terminal:

->piece initial state [4, 4, 6]

->piece to state [3, 4, 6]

STATE COMPROBADO: [[3, 4, 6], [7, 0, 2]]

JAUQUE MATE: False

R moved.

->piece initial state [7, 0, 2]

->piece to state [0, 0, 2]

STATE COMPROBADO: [[0, 0, 2], [4, 4, 6]]

JAUQUE MATE: False

K moved.

->piece initial state [3, 4, 6]

->piece to state [2, 4, 6]
STATE COMPROBADO: [[2, 4, 6], [7, 0, 2]]

Lo que me preocupaba de inicio es esta salida de terminal:

```
JAQUE MATE: False
There is no piece to move at the start place
STATE COMPROBADO: [[7, 4, 2], [4, 3, 2]]
JAQUE MATE: False
There is no piece to move at the start place
STATE COMPROBADO: [[4, 2, 2], [4, 3, 2]]
JAQUE MATE: False
There is no piece to move at the start place
STATE COMPROBADO: [[4, 1, 2], [4, 3, 2]]
JAQUE MATE: False
There is no piece to move at the start place
STATE COMPROBADO: [[4, 0, 2], [4, 3, 2]]
JAQUE MATE: False
There is no piece to move at the start place
STATE COMPROBADO: [[4, 5, 2], [4, 3, 2]]
JAQUE MATE: False
There is no piece to move at the start place
STATE COMPROBADO: [[4, 6, 2], [4, 3, 2]]
JAQUE MATE: False
There is no piece to move at the start place
STATE COMPROBADO: [[4, 7, 2], [4, 3, 2]]
JAQUE MATE: False
#A* move sequence: []
A* End
```

Donde lo que veo es que en algún punto del recorrido estrella pierde o se cambia el rey blanco por otra torre blanca porque el estado devuelve una lista con dos identificadores 2, como si hubieran dos torres, posiblemente la causa de que el código nunca encuentra la solución debido a la desaparición del rey con identificador 6, es la teoría inicial que tengo.

Primera modificación de código hice un comprobar si el rey blanco continúa en tablero sino descartar este posible movimiento para evitar esa pérdida:

```
if self.getPieceState(movimiento, 6) is None:
    continue
```

Y parece funcionar de primera para ese primer problema observado

```
STATE COMPROBADO: [[5, 3, 2], [3, 4, 6]]
JAQUE MATE: False
R moved.
->piece initial state [5, 3, 2]
->piece to state [6, 3, 2]
STATE COMPROBADO: [[6, 3, 2], [4, 4, 6]]
```

```
JAQUE MATE: False
There is no piece to move at the start place
STATE COMPROBADO: [[6, 3, 2], [3, 4, 6]]
JAQUE MATE: False
#A* move sequence: []
A* End
```

Como se ve en la nueva salida de terminal, el final aunque no encuentra solución al menos ya conserva la pieza del rey blanco.

Segundo cambio que implemente fue evitar el problema que se coman al rey negro:

```
if movimiento[0][0:2] == [0,4]:
    continue
```

Funciona porque ahora imprimiendo el tablero ahora al final el rey negro se conserva pero sigue sin encontrar la solución.

Nuevos cambios:

Tras unos días sin saber solución, encontré que la forma para poder encontrar la solución es dejar de usar el método `movePieces()` y en su lugar reconstruir el tablero simulando con `newBoardSim` del estado actual mas recordado de nuevo la posición del rey porque si pasaba únicamente el state del momento se perdía el rey negro en el tablero

```
self.newBoardSim(self.copyState(state) + [[0, 4, 12]])
```

Con esto y cambiando el inicio que empezara a 0, me di cuenta que mi forma de cálculo del coste añadía un +1 innecesario y que de hecho aplicaba mal porque sumaba un mas uno pero luego para los cálculos usaba otra variable de cálculo anterior provocando que tenía líneas innecesarias. aplique y elimine lo pertinente y comprobé que al final el coste ahora si estaba bien por los movimientos hechos.

Tras estas correcciones volví a implementar imprimir el tablero en cada movimiento y vi que ahora el código funcionaba tanto para un caso actual como para el caso anterior de las piezas y no perdía ninguna pieza y me devolvió una solución correcta para el problema.

Declaración de IA

La IA de Gemini principalmente para la información y aprendizaje del formato de algoritmo de búsqueda en estrella y la comprensión de las funciones que me dan en el código de aichess.

Le pedí que me mostrara ejemplos de algoritmos de estrella básicos, y que analizara el código de aichess para explicarme bien que hace cada función.

Para el primer apartado yo hice un código y le pedí que analizará y me mostrará los errores que había cometido porque había momentos en los que no sabía por qué no me funcionaba y la IA me ayudaba con detalles teóricos que no tenía en cuenta.

Un ejemplo claro de esto es en el apartado 2, donde yo en mi código había analizado el error de que no encontraba una solución debido a que en algún momento se desincronizaba los movimientos, y gracias a la IA fue que me explicó por qué sucedía esto.

Debido a que por una razón que no sé explicar, la función de mover piezas provocaba una especie de desincronización del tablero y me recomendó en lugar de usar mover piezas, usar New Board Sim, o sea, volver a recrear el tablero en cada movimiento.

Básicamente mi trabajo de código lo he hecho yo, pero cuando llegaba a un camino sin salida que ya no sabía qué más hacer, o yo me daba cuenta de un error, pero que no sabía cómo arreglarlo, recurrí a la IA para que me dijera qué es lo que estaba pasando y qué es lo que tenía que hacer a partir de mis ideas/procesos que había observado/intentado arreglar, pero la IA en ningún momento me ha modificado mi código. Todo ha sido aconsejamiento. He usado la IA como un profesor auxiliar de práctica para comprender mejor la práctica y encontrar salidas que no lograba encontrar por mi cuenta.